

Documentação de Banco de Dados - Solidarity Connection

Grupo 09

Ana Clara Marson Rios Bordinhon – RM367890

Daiana Silveira Russi Firmino – RM367375

Contexto:

Nossa aplicação foi pensada principalmente para oferecer fluidez na navegação e clareza nas transações realizadas.

Pensando em como a aplicação seria utilizada em produção, decidimos por "dividi-la" em dois modelos distintos de acesso a dados: transações de doações (escrita, consistência forte) vs consulta pública de transparência (leitura massiva, dados agregados). Utilizar o CQRS (*Command Query Responsibility Segregation*) para realizar a separação escrita x leitura traz diversos benefícios de performance e escalabilidade independentes. Com diferentes modelos, temos diferentes necessidades e diferentes ferramentas para atendê-las, onde decidimos por seguir com **SQL Server** e **MongoDB**.

Modelo de Escrita/Leitura: SQL Server

- ACID: A consistência ACID (atomicidade, consistência, isolamento e durabilidade) é essencial para uma aplicação robusta, principalmente quando falamos de dinheiro/doações: rastreabilidade completa dos dados e possibilidade de *rollback* total da transação – sabemos que não se pode haver inconsistência no saldo de uma campanha, por exemplo.
- Relacionamentos bem definidos (Usuário — nos papéis de Doador e Gestor de ONG —, Campanha, Doação) se beneficiam de *schema* relacional e integridade referencial.
- EF Core: O SQL Server nos permite aproveitar recursos do EF Core como o *OwnsOne*, usado para mapear nossos *Value Objects* (ex. CPF) diretamente como parte da entidade dona, sem a necessidade de tabelas adicionais só para representar conceitos que, na prática, sempre existem atrelados a uma entidade principal. Esse nível de mapeamento reforça o motivo de mantermos essa camada em um banco relacional: ele foi desenhado para lidar com esse tipo de relacionamento estruturado.
- É importante destacar que essa separação de responsabilidades acontece a nível de código, e não significa que o SQL Server só realiza escritas. Ele continua sendo usado também para leitura em fluxos como autenticação e consulta de campanhas — o MongoDB entra especificamente como um modelo de leitura dedicado para o painel de transparência, onde o ganho de desempenho e desacoplamento justificava essa camada extra.

Modelo de Leitura: MongoDB

- Leitura de alto volume/baixa latência para consulta pública, sem impactar o banco transacional.
- Para o *dashboard*, é importante que os dados já venham praticamente prontos – agregados e desnormalizados – evitando a necessidade de múltiplos relacionamentos em diversas tabelas para chegar-se ao resultado final.
- Desacoplar o modelo de leitura do modelo de escrita facilita evoluções e mudanças sem múltiplos impactos. Além disso, por possuir um formato próximo ao *JSON*, facilita a utilização dos dados pelo *front-end*.

- O MongoDB nos permite flexibilidade na mudança de *schemas* de um documento, sem necessidade de *migrations* – basta os novos campos serem enviados durante a integração.

Trade-offs

Essa separação, no entanto, não vem sem custos — algumas consequências precisam ser consideradas, como:

- Consistência eventual entre os dois bancos (não é consistência forte imediata)
- Complexidade operacional extra de manter dois bancos

Porém, entendemos que uma aplicação bem estruturada, mesmo que adicionando certa complexidade, pode nos trazer mais segurança e estabilidade no dia a dia. A consistência eventual também não é um grande problema, visto que dentro da aplicação, os processos já são assíncronos, com filas, onde precisamos aguardar o processamento externo do pagamento para efetivar a operação realizada.

Conclusão

Ao longo do desenvolvimento, ficou claro pra gente que forçar um único banco a atender tanto a escrita transacional quanto a leitura pública do dashboard ia gerar gargalos e acoplamento desnecessário. Por isso, dividimos as responsabilidades: o **SQL Server** continua sendo responsável tanto pela escrita quanto pela leitura das regras de negócio centrais (autenticação, campanhas, doações), enquanto o **MongoDB** entrega uma projeção de leitura dedicada exclusivamente ao painel de transparência. O **CQRS**, nesse caso, foi aplicado de forma seletiva: não como uma separação total entre bancos de escrita e leitura, mas como uma camada extra de leitura otimizada apenas onde o ganho de desempenho e desacoplamento realmente compensava a complexidade adicional.

Essa projeção é atualizada de forma assíncrona por um *consumer* de eventos que roda dentro da própria API, escutando a fila *donation-processed* publicada pelo *Worker* após o processamento do pagamento. Optamos por manter esse *consumer* no mesmo processo da API porque ele já precisa acessar o contexto do SQL Server para atualizar o status da doação e da campanha antes de projetar essa informação no MongoDB — evitando uma conexão redundante ao banco transacional em outro processo. Essa não foi uma decisão arbitrária — é o CQRS aplicado na prática, na camada de dados da nossa arquitetura.

Essa decisão também fica evidente na prática: o *endpoint* já existente *GET /api/campaign/active*, que consulta diretamente o SQL Server, permaneceu intacto. Para o painel de transparência, criamos um *endpoint* separado, *GET /api/campaign/transparency-dashboard*, consultando exclusivamente o MongoDB. Dessa forma, a evolução do *dashboard* não impacta a lógica de negócio já existente e conseguimos evoluir cada modelo de forma independente.